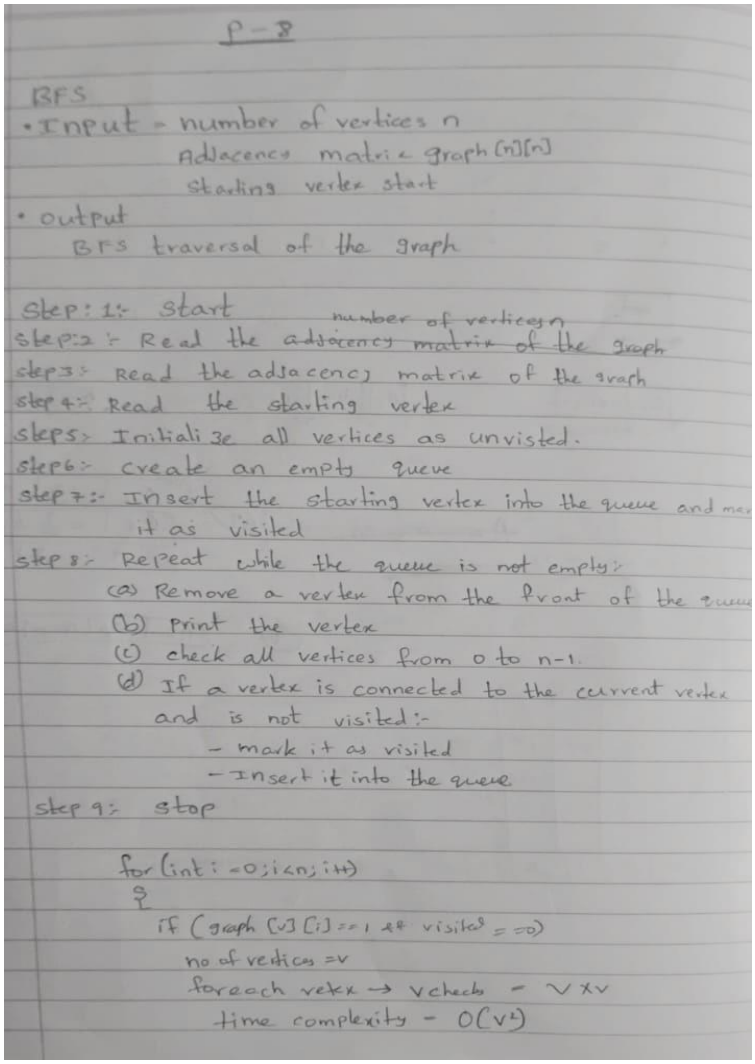
PRACTICAL - 8

AIM :Implementation of Graph and Searching (DFS and BFS)

Method 1 :BFS

Algorithm :



P-8

BFS
• Input - number of vertices n
Adjacency matrix graph[n][n]
Starting vertex start
• output
BFS traversal of the graph

step 1:- Start
step 2:- Read the adjacency matrix of the graph number of vertices n
step 3:- Read the adjacency matrix of the graph
step 4:- Read the starting vertex
step 5:- Initialize all vertices as unvisited.
step 6:- create an empty queue
step 7:- Insert the starting vertex into the queue and mar
it as visited
step 8:- Repeat while the queue is not empty:
(a) Remove a vertex from the front of the queue
(b) Print the vertex
(c) check all vertices from 0 to n-1.
(d) If a vertex is connected to the current vertex
and is not visited:-
- mark it as visited
- Insert it into the queue
step 9:- stop

for(int i=0;i<n;i++)
{
if (graph[v][i]==1 && visited==0)
no of vertices =v
foreach vertex → vchecks = v x v
time complexity - O(v²)

code :

```
#include <iostream>
using namespace std;

int main()
```

```
{
   int n, graph[10][10], visited[10] = {0};
   int queue[10], front = 0, rear = 0;
   int start;

   cout << "Enter number of vertices: ";
   cin >> n;

   cout << "Enter adjacency matrix:\n";
   for(int i = 0; i < n; i++)
      for(int j = 0; j < n; j++)
         cin >> graph[i][j];

   cout << "Enter starting vertex: ";
   cin >> start;

   queue[rear++] = start;
   visited[start] = 1;

   cout << "BFS: ";

   while(front < rear)
   {
      int v = queue[front++];
      cout << v << " ";

      for(int i = 0; i < n; i++)
      {
         if(graph[v][i] == 1 && visited[i] == 0)
         {
            queue[rear++] = i;
            visited[i] = 1;
         }
      }
   }

   return 0;
}
```

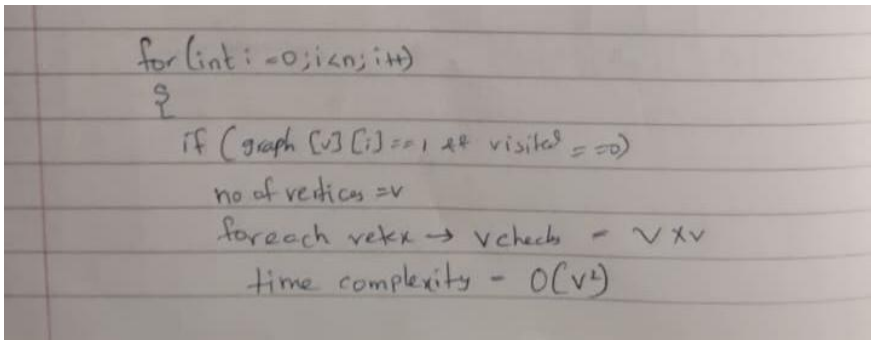
Output :

```
Output

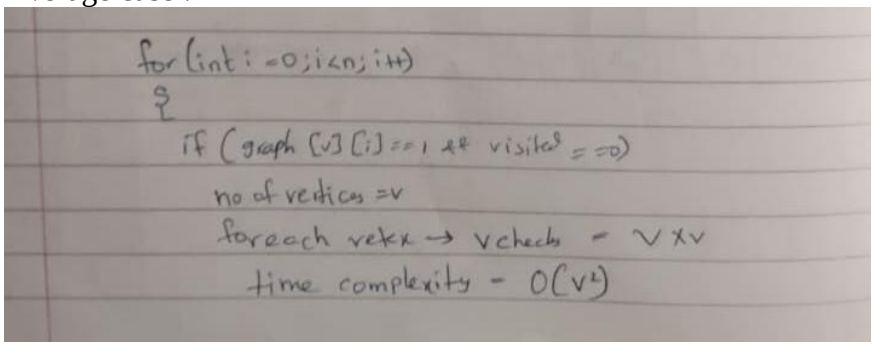
Enter number of vertices: 5
Enter adjacency matrix:
0 1 1 0 0
1 0 0 1 1
1 0 0 0 1
0 1 0 0 0
0 1 1 0 0
Enter starting vertex: 0
BFS: 0 1 2 3 4
```

Time complexity :

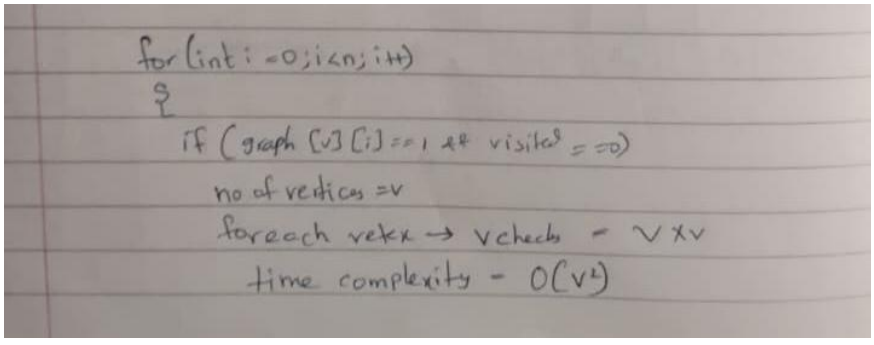
Best case :



Average case :

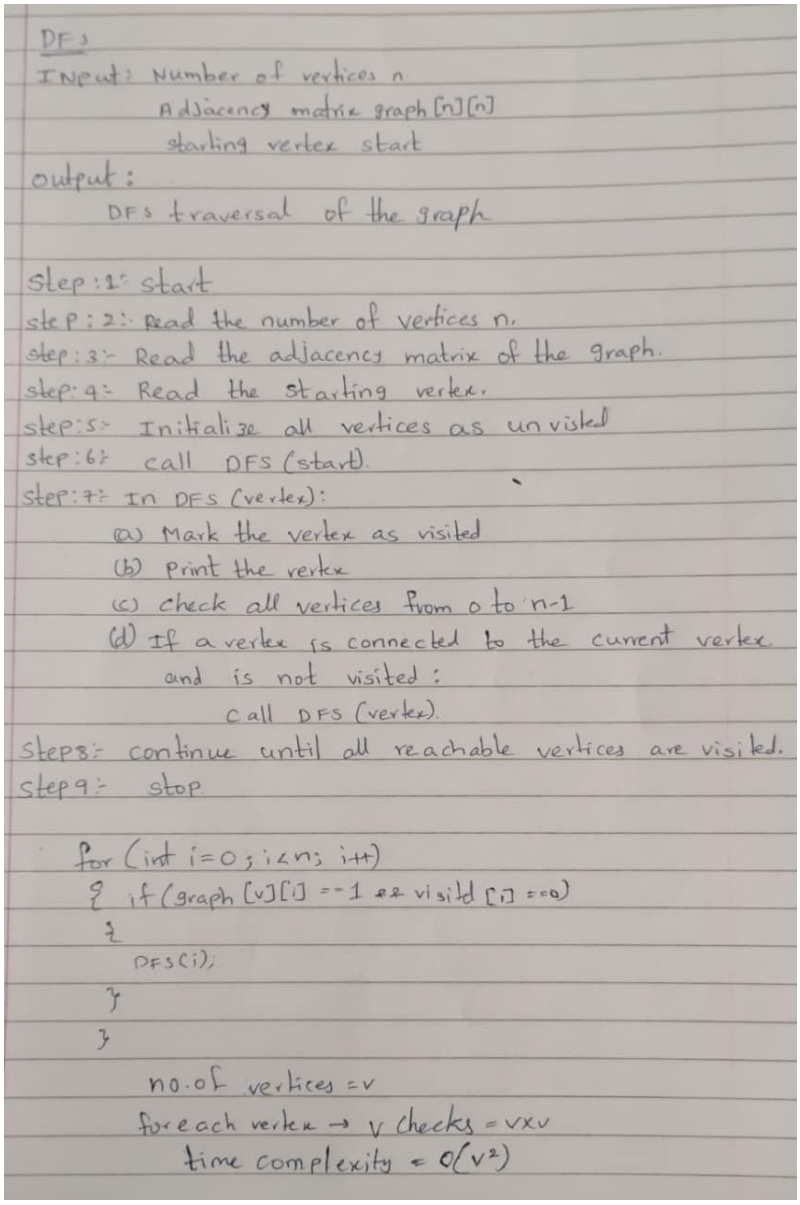


Worst case :



Method 2 : DFS

Algorithm :



Code :

```
#include <iostream>
using namespace std;
```

```
int graph[10][10];
int visited[10] = {0};
int n;

void DFS(int v)
{
   cout << v << " ";
   visited[v] = 1;

   for(int i = 0; i < n; i++)
   {
      if(graph[v][i] == 1 && visited[i] == 0)
      {
         DFS(i);
      }
   }
}

int main()
{
   int start;

   cout << "Enter number of vertices: ";
   cin >> n;

   cout << "Enter adjacency matrix:\n";
   for(int i = 0; i < n; i++)
      for(int j = 0; j < n; j++)
         cin >> graph[i][j];

   cout << "Enter starting vertex: ";
   cin >> start;

   cout << "DFS: ";
   DFS(start);

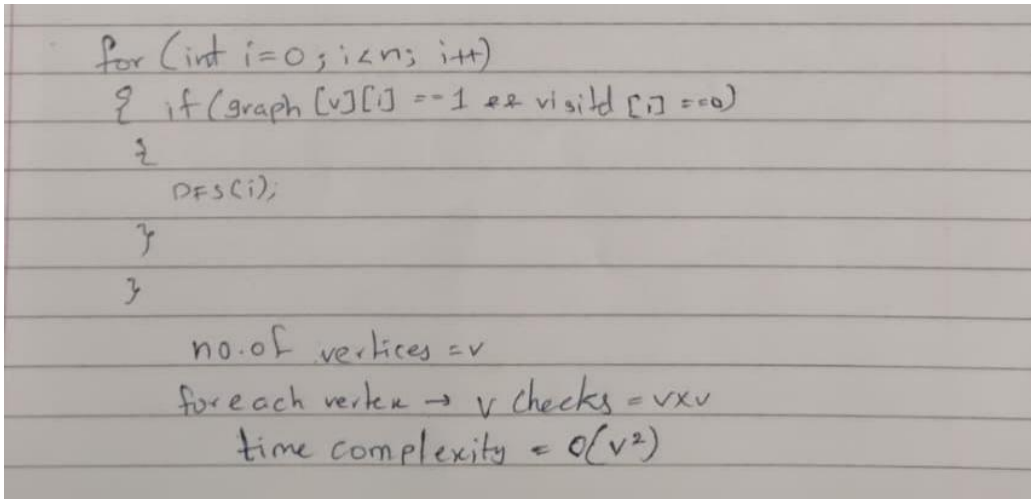
   return 0;
}
```

Output :

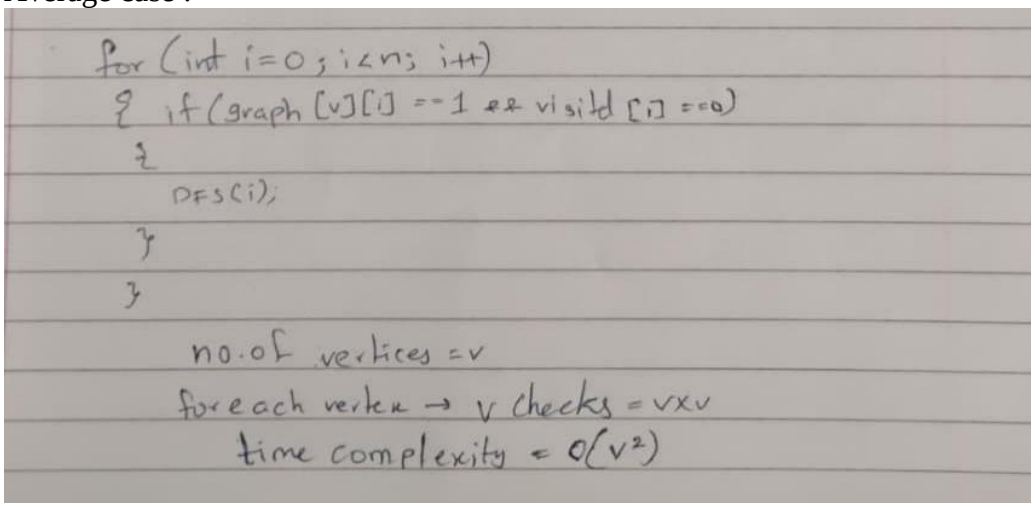
```
Enter number of vertices: 5
Enter adjacency matrix:
0 1 1 0 0
1 0 0 1 1
1 0 0 0 1
0 1 0 0 0
0 1 1 0 0
Enter starting vertex: 0
DFS: 0 1 3 4 2
```

Time complexity :

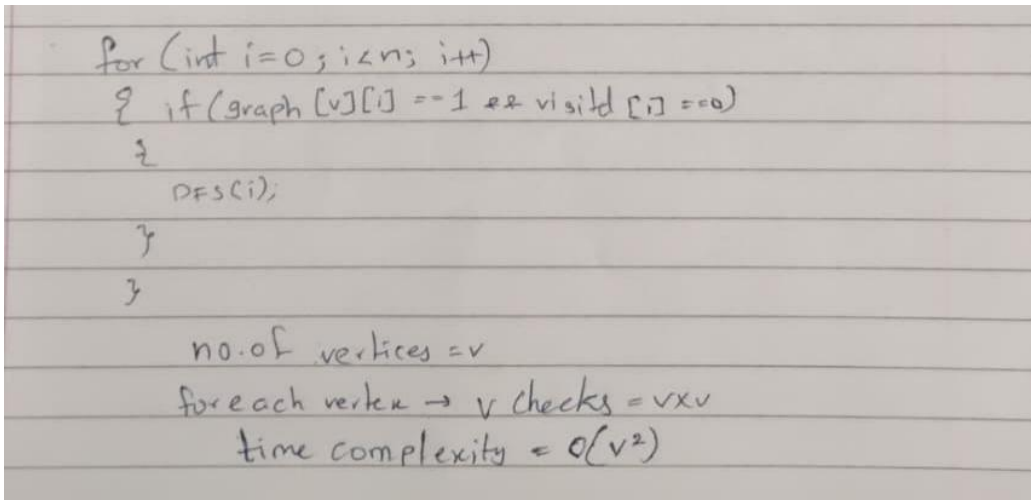
Best case :



Average case :



Worst case :



Conclusion :

The Making Change Problem was successfully implemented using Dynamic Programming. The algorithm efficiently finds the minimum number of coins required for a given amount with a time complexity of O(nW).